

# Komorebi

Prediction of olive phenological states  
for optimal harvest time

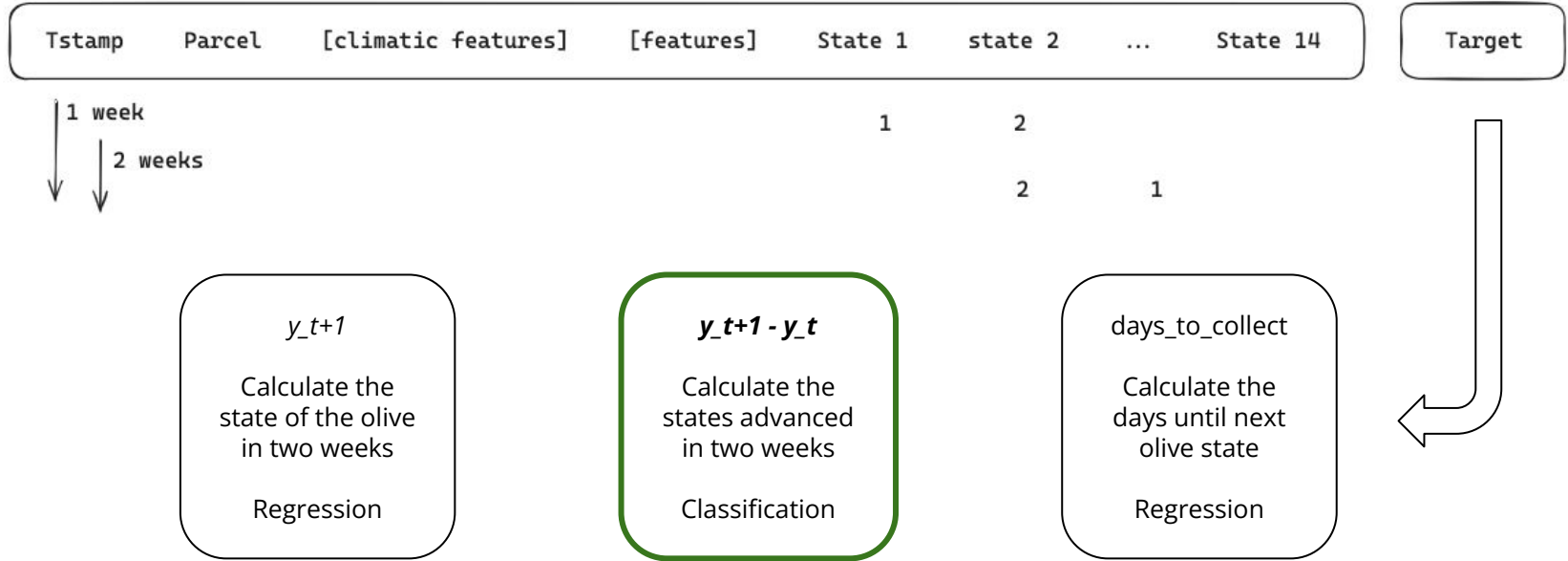
Zrive 1Q24

# Agenda

- Problem statement
- Modelling
- Features
- Baselines and ML models
- Results
- Next steps

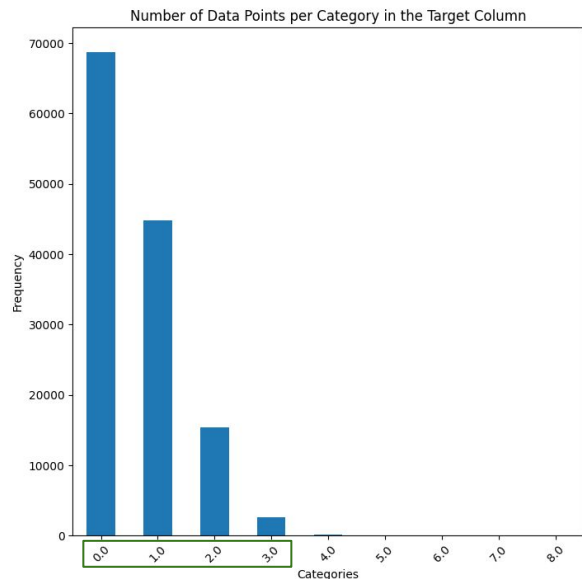
# Problem statement

Predicting the maturation stage of the olive within two weeks



# Modelling

## Target variable classes



## Metrics

### Accuracy

Percentage of total predictions that exactly match the actual stages of olive advancement

### MSE

Average of the squared differences between the predicted and actual stages

### Precision\_for\_class

Accuracy of predictions comparing true positives to all predicted positives for each specific stage

### Recall\_for\_class

How well the model identifies all actual cases of a specific stage of olive advancement

# Modelling

## EDAs

Key insights:

- There are 5 years in which we have data from the complete harvest in both datasets.
- All harvests seem to be independent.

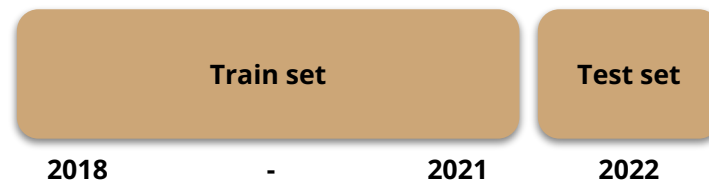
Key actions for the parcels dataframe:

- Remove rows with null phenological states.
- Remove parcels whose codes appeared in multiple provinces.
- Remove highly spaced samples.
- Remove parcels where we don't have meteo data.

Key actions for the meteo dataframe:

- Combine indices into a single category.
- Drop NaNs from index column.
- Remove rows with 0.0 as value.

## Train/Test split



- Strong temporal component in this problem
- We need one entire harvest to test
- Ideally:
  - We would have a validation set
  - The test set would include different types of years (humid season, dry, etc...)

# Modelling

## Target function

```
def generate_target(parcelas_df:pd.DataFrame, window_size:int=14, window_tolerance:int=2)->pd.DataFrame:
    df = parcelas_df[TARGET_COLS].copy()
    df['fecha'] = df['fecha'].astype('datetime64[ns]') # Establish units in datetime

    # Generate date to search PHENOLOGICAL_STATE in the future
    parcelas_df['fecha_futuro'] = parcelas_df['fecha'] + pd.Timedelta(days=window_size)

    # Join between the 2 dfs with a time delta (±2 days)
    parcelas_with_target_df = pd.merge_asof(
        parcelas_df.sort_values('fecha'), df.sort_values('fecha'),
        by='codparcela', left_on='fecha_futuro', right_on='fecha',
        suffixes=('', '_future'), direction='nearest', tolerance=pd.Timedelta(days=window_tolerance))

    # Generate target column - Number of phenological states that passed in the chosen window size
    parcelas_with_target_df['target'] = parcelas_with_target_df['estado_mayoritario_future'] - parcelas_with_target_df['estado_mayoritario']

    # Drop all rows with NULL in target
    parcelas_with_target_df = parcelas_with_target_df[parcelas_with_target_df['target'].notnull()]

    # Impute to 0 when target column is negative
    parcelas_with_target_df.loc[parcelas_with_target_df['target'] < 0, 'target'] = 0

    return parcelas_with_target_df.sort_values(by=['codparcela', 'fecha'])
```

# Features

Modelled using only numerical variables

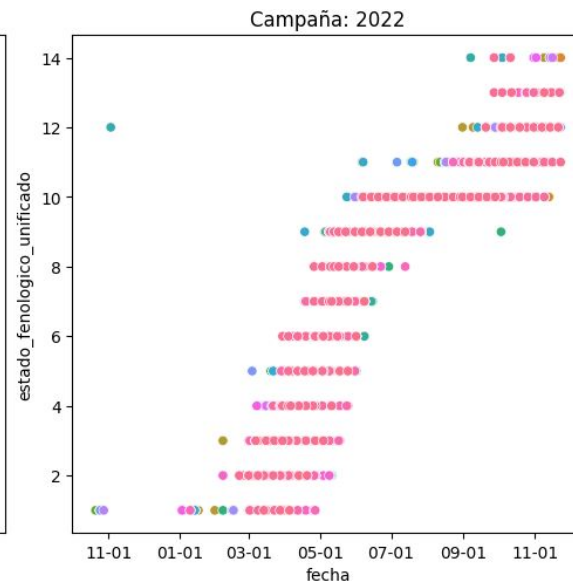
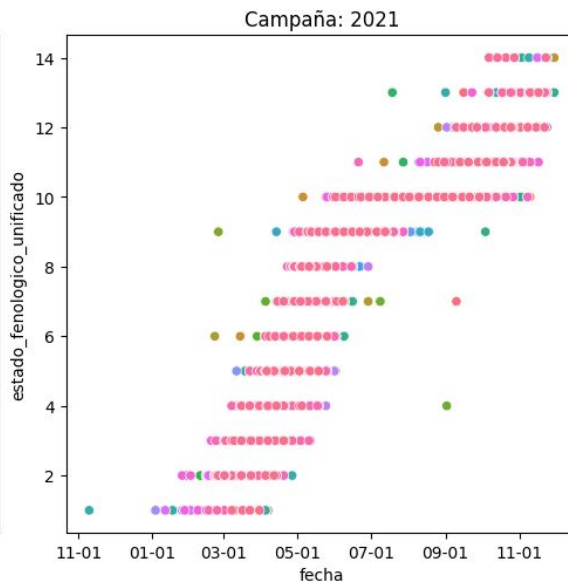
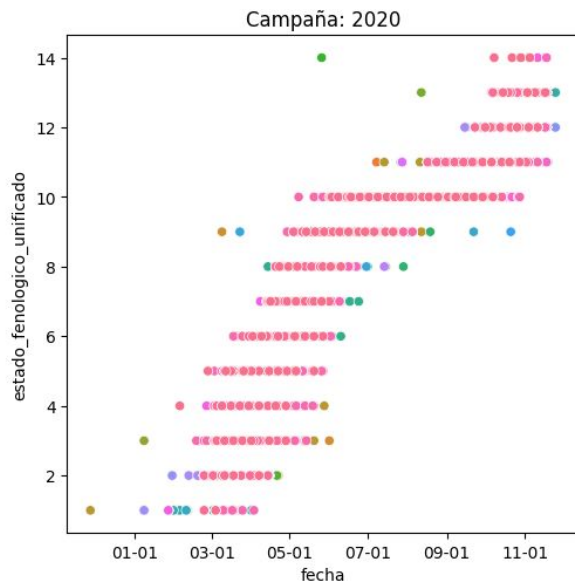
Left categorical variables for future exploring due to high cardinality

Generated features:

- Unified phenological state
- Week number
- Days in current state
- Total days in previous state
- Descriptive statistics of meteorological variables

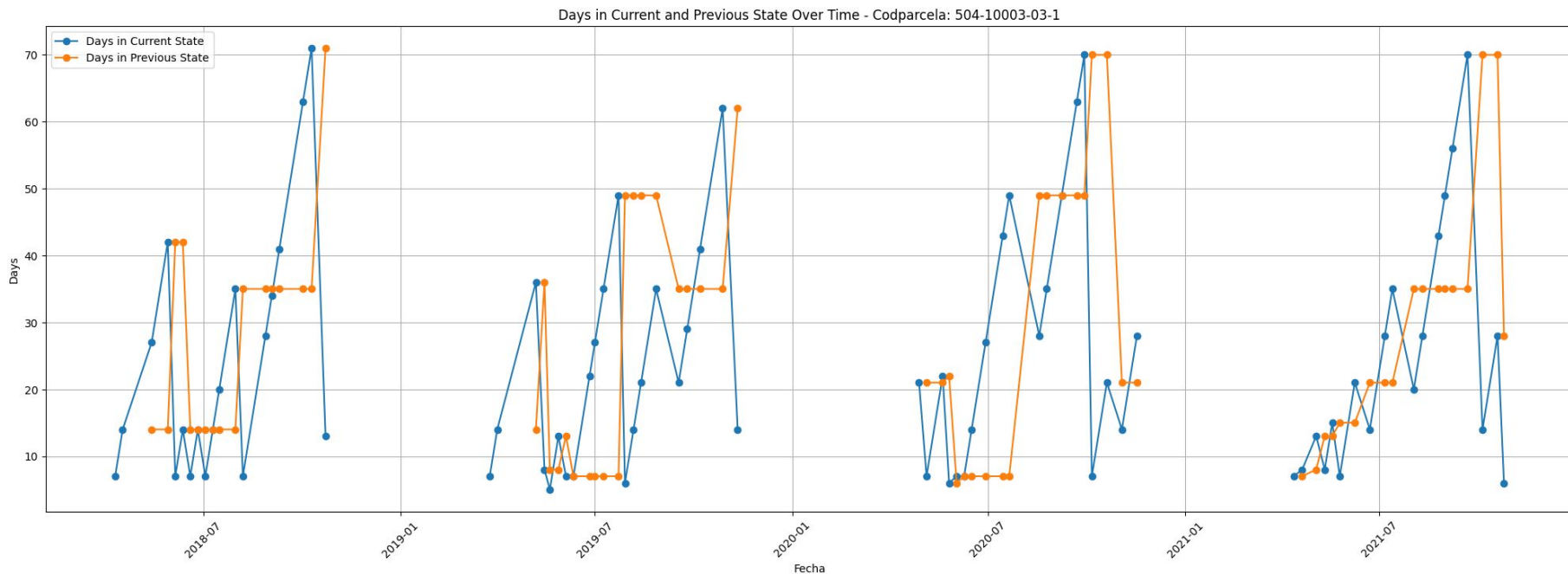
# Features

Unified phenological state → Week number



# Features

Days in current state & Total days in previous state



# Features

Descriptive statistics of meteorological variables

Count, mean, std, median, min, max  
Variable rolling period: 30d, 90d, 365d

```
def attach_meteo_var(  
    spine_df:pd.DataFrame, meteo_df:pd.DataFrame, columns_to_attach:list, window_tolerance:int=2  
)->pd.DataFrame:  
  
    total_df = pd.merge_asof(  
        spine_df.sort_values('fecha'), meteo_df[['codparcela', 'fecha'] + columns_to_attach].sort_values('fecha'),  
        by='codparcela', left_on='fecha', right_on='fecha',  
        direction='backward', tolerance=pd.Timedelta(days=window_tolerance))  
  
    return total_df  
  
def calculate_climatic_stats_time_window(meteo_df:pd.DataFrame, rolling_window:str='30D')->pd.DataFrame:  
    # Format dataframe  
    stat_df = meteo_df[['codparcela','fecha'] + METEEO_COLUMNS].copy()  
    stat_df['fecha'] = pd.to_datetime(stat_df['fecha'])  
    stat_df.sort_values(by=['codparcela', 'fecha'], inplace=True)  
    stat_df.set_index('fecha', inplace=True)  
  
    # Calculate descriptive statistics  
    stat_df = stat_df.groupby('codparcela').rolling(rolling_window).agg(['count','mean','std','min','median','max'])  
    stat_df.columns = ['_'.join(col) + '_' + rolling_window for col in stat_df.columns]  
    stat_df = stat_df.reset_index()  
  
    return stat_df
```

# Features

Descriptive statistics of meteorological variables

Count, mean, std, median, min, max  
Variable rolling period: 30d, 90d, 365d

| codparcela      | fecha      | target | FAPAR_count_30D | FAPAR_mean_30D | FAPAR_std_30D | FAPAR_min_30D | FAPAR_median_30D | FAPAR_max_30D | GNDVI_count_30D | ... | SIPI_std_365D | SIPI_min_365D | SIPI_median_365D | SIPI_max_365D |
|-----------------|------------|--------|-----------------|----------------|---------------|---------------|------------------|---------------|-----------------|-----|---------------|---------------|------------------|---------------|
| 012-00002-00-GD | 2018-01-15 | 2.0    | 1.0             | 0.498039       | NaN           | 0.498039      | 0.498039         | 0.498039      | 1.0             | ... | NaN           | 0.914284      | 0.914284         | 0.914284      |
| 007-00011-01-00 | 2018-03-01 | 0.0    | 4.0             | 0.699020       | 0.206057      | 0.537255      | 0.629412         | 1.000000      | 3.0             | ... | 0.344929      | 0.372233      | 1.117407         | 1.197304      |
| 029-00081-06-00 | 2018-03-05 | 1.0    | 3.0             | 0.070588       | 0.011765      | 0.058824      | 0.070588         | 0.082353      | 3.0             | ... | 0.344929      | 0.372233      | 1.117407         | 1.197304      |
| 001-00004-02-00 | 2018-03-05 | 1.0    | 3.0             | 0.253595       | 0.045452      | 0.211765      | 0.247059         | 0.301961      | 3.0             | ... | 0.344929      | 0.372233      | 1.117407         | 1.197304      |
| 003-00058-02-00 | 2018-03-05 | 1.0    | 3.0             | 0.411765       | 0.006792      | 0.403922      | 0.415686         | 0.415686      | 3.0             | ... | 0.344929      | 0.372233      | 1.117407         | 1.197304      |

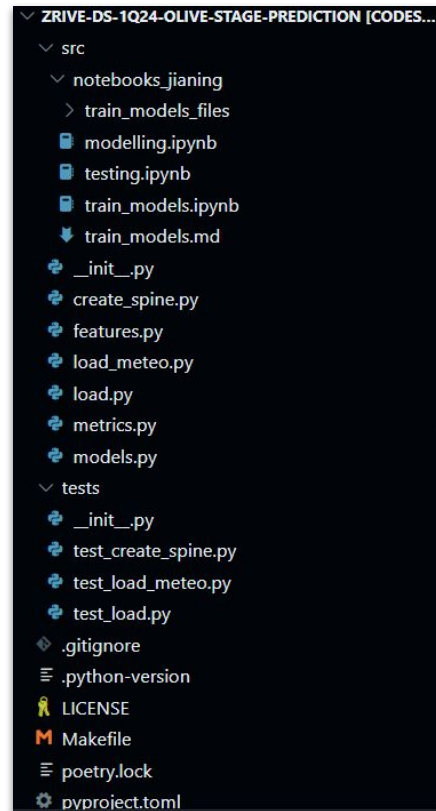
# Baseline and ML models

## Baseline

*Baseline Prediction = round(WeightedMean(y | phenological state, year))*

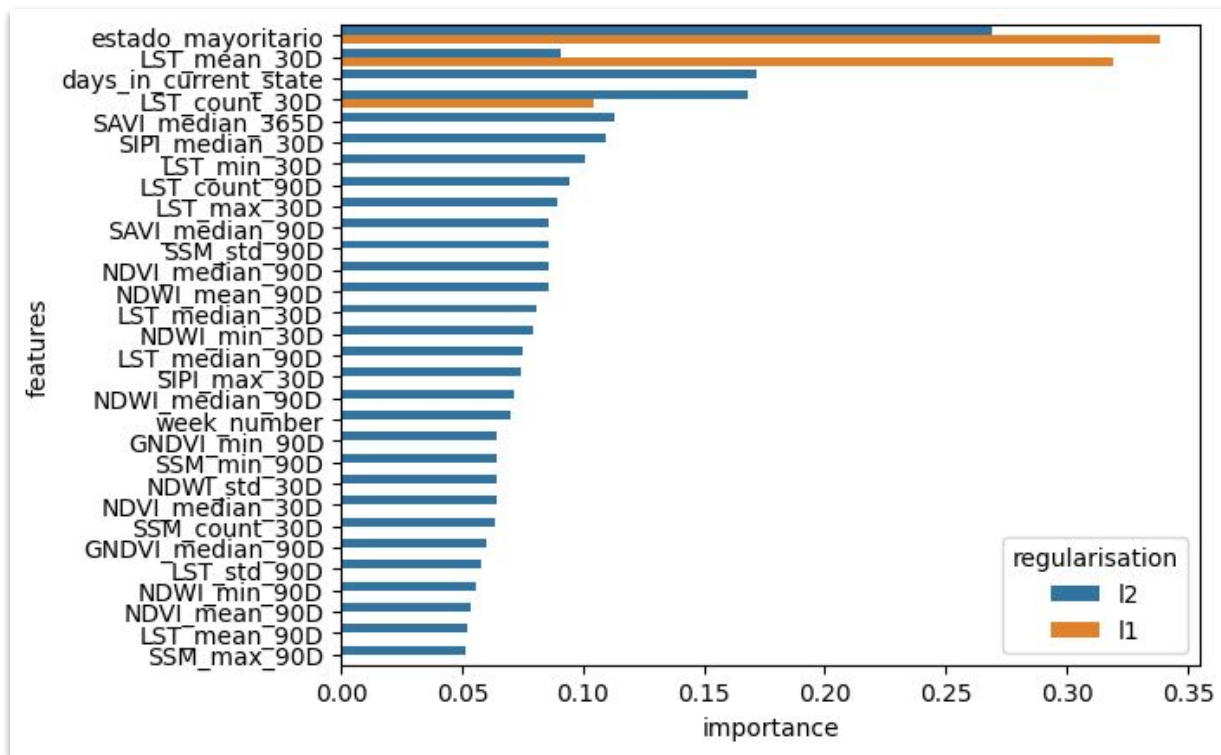
## ML Models:

- Logistic Regression
- Random Forest
- LightGBM
- XGBoost

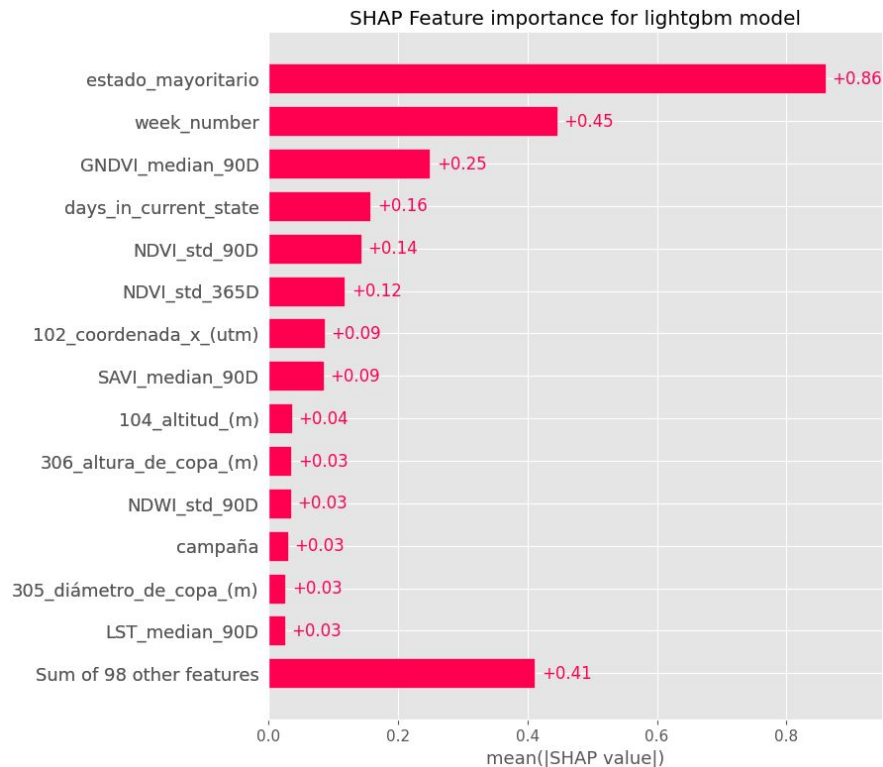


# Baseline and ML models

## Feature importance of Logistic Regression models



# Feature importance with SHAP

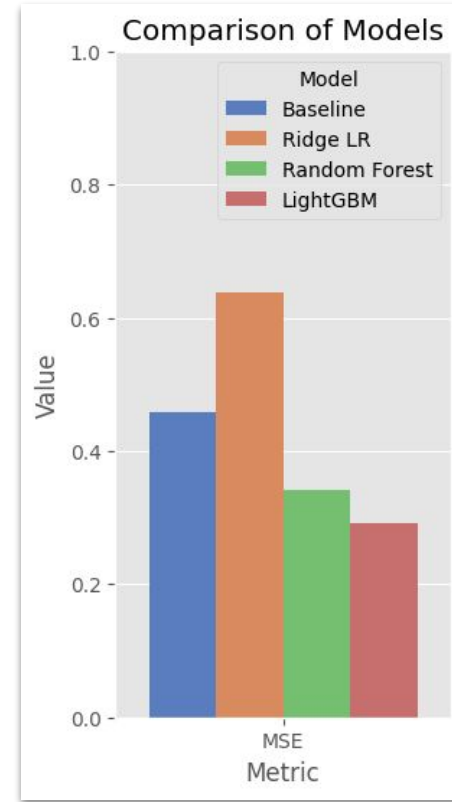
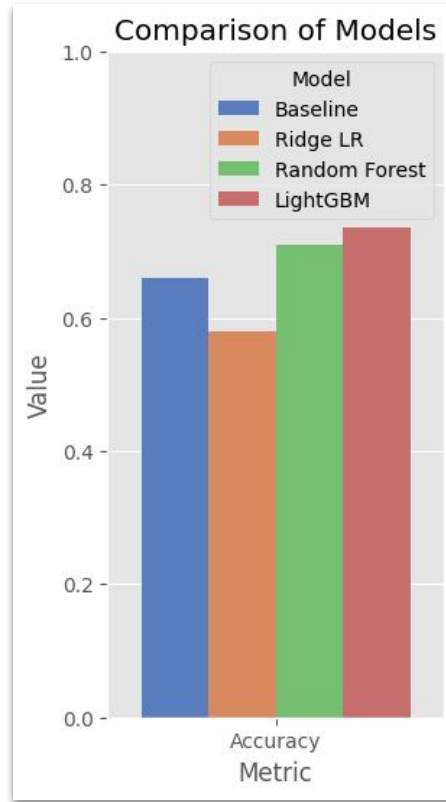


# Results

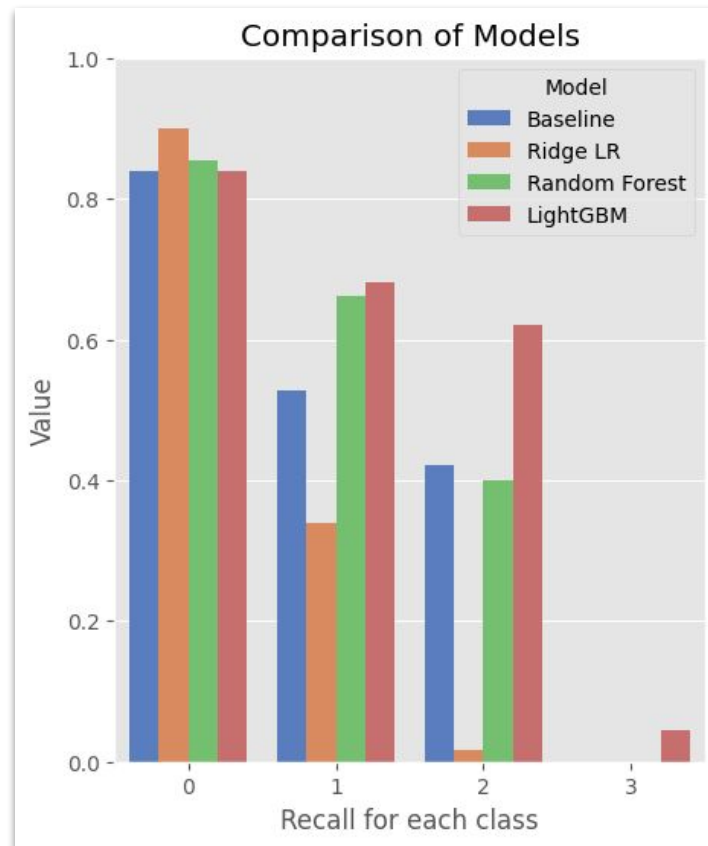
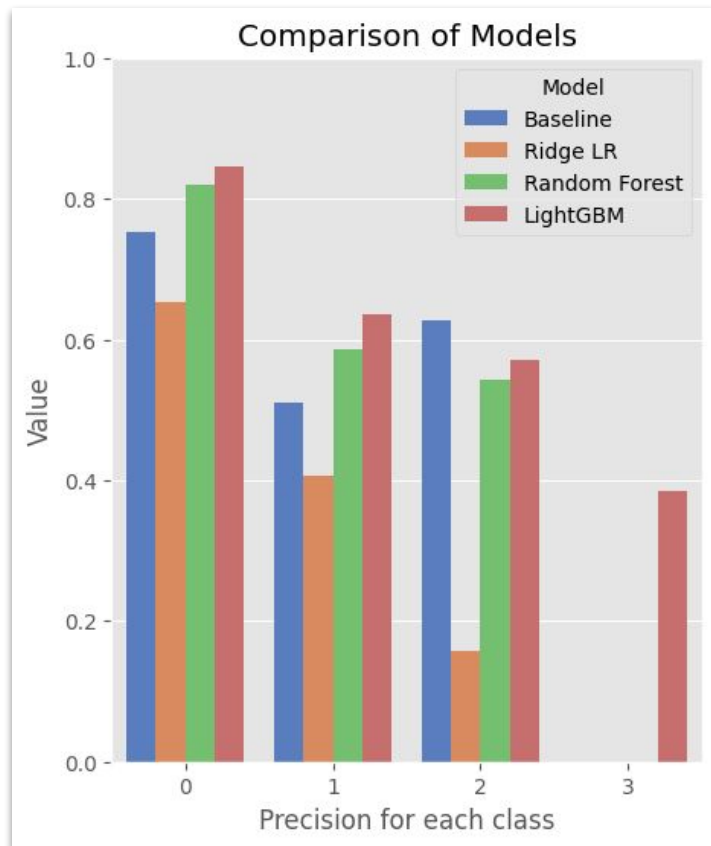
Evaluation metrics used:

- Accuracy
- MSE
- Precision for each class
- Recall for each class

Non-linear models beat the baseline!

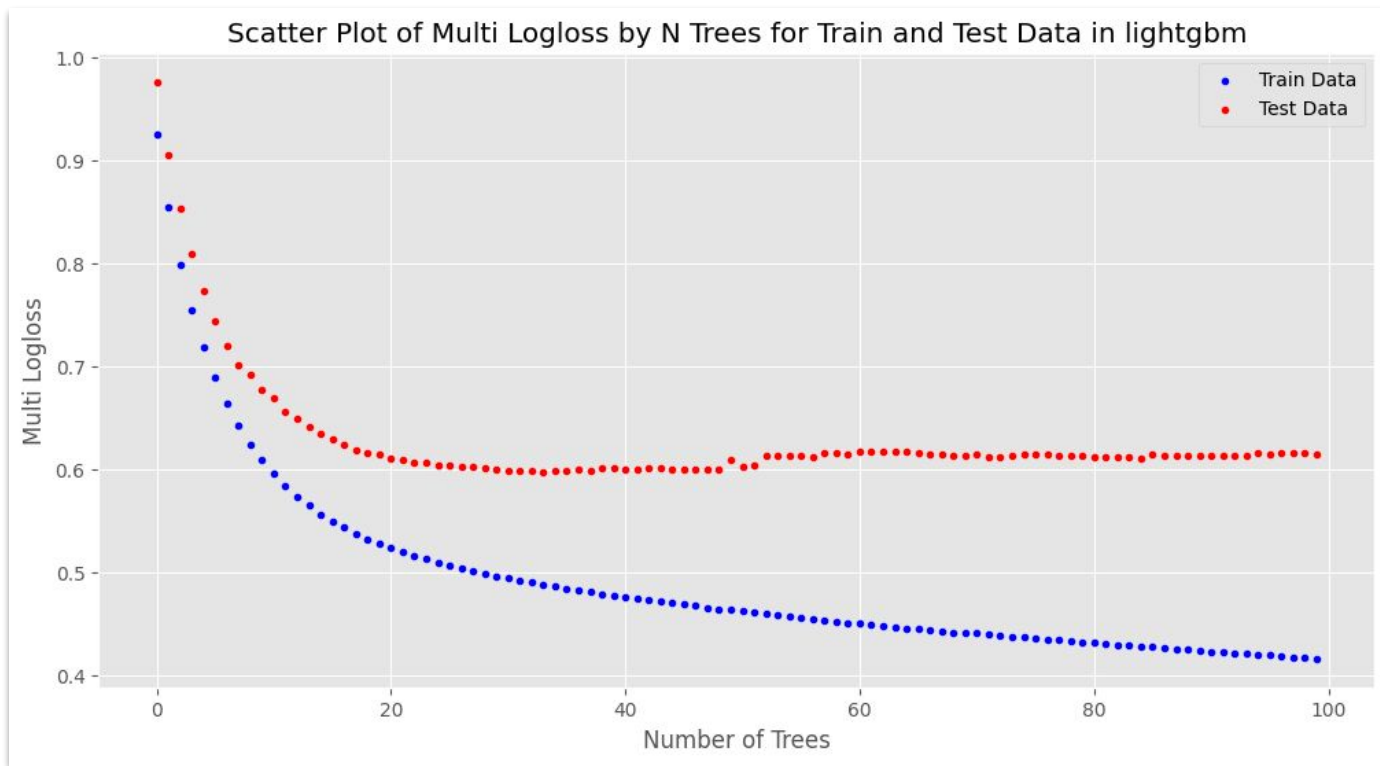


# Results



# Results

Is the model learning or generalizing?



# Next steps

- Experiment with categorical features
- Create more features
- Utilize more data
- Model the problem as a regression
- Try different options to impute NaNs

# Thank you

Q&A

# 6-7 diapos

Problem statement (Alvaro)

Como modelar el problema - EDAS, Clasificación, métricas, train/test, etc...

Modelos (Target, baseline)

Features (Alejandro)

Resultados (Jianing)

Next steps: Cosas que nos hemos dejado en el tintero: Incluir el nuevo dataset, meterse con las categoricas, probar regresión, etc...